

Exposure Fusion

Studio e implementazione in Python

Analisi del paper, mappatura sul codice e risultati sperimentali



Documento tecnico sull'implementazione dell'algoritmo di Exposure Fusion di Mertens, Kautz e Van Reeth, con formule, corrispondenza al codice e risultati su dataset di esposizioni multiple.

Progetto realizzato per il corso di Computer Graphics della Laurea Magistrale in Informatica, tenuto dal Prof. Fabio Pellacini presso l'Università degli Studi di Modena e Reggio Emilia (Unimore).

Angelo Longano

Laurea Magistrale in Informatica - Unimore

7 giugno 2026

Indice

1	Abstract	2
2	Introduzione	2
3	Metodo	3
4	Struttura dell'implementazione	4
5	Calcolo delle mappe di peso	5
5.1	Contrasto	5
5.2	Saturazione	5
5.3	Well-exposedness	5
6	Normalizzazione dei pesi	6
7	Fusione multirisoluzione	7
8	Dataset e risultati	8
8.1	Venice Boat	8
8.2	Venice Carnival	9
8.3	Living Room Window	10
9	Controlli di correttezza	10
9.1	Normalizzazione	11
9.2	Ricostruzione della piramide Laplaciana	11
9.3	Coerenza delle shape	11
10	Differenze implementative e limiti	12
11	Conclusioni	12
12	Riferimenti	13

1 Abstract

Questo documento presenta lo studio e l'implementazione in Python dell'algoritmo **Exposure Fusion** proposto da Mertens, Kautz e Van Reeth. Il metodo combina una sequenza di immagini LDR acquisite con esposizioni differenti in una singola immagine LDR finale, senza ricostruire una radiance map HDR e senza richiedere stima della Camera Response Function o tone mapping.

L'implementazione realizzata riproduce il nucleo algoritmico del paper: per ogni immagine vengono calcolate misure locali di qualità, queste misure vengono combinate in mappe di peso normalizzate e la fusione finale viene eseguita in multirisoluzione tramite piramidi Laplaciane delle immagini e piramidi Gaussiane dei pesi. Il progetto non mira a essere una replica bit-exact del codice originale degli autori, ma a rendere esplicita la corrispondenza tra formule, codice Python e risultati visivi.

2 Introduzione

Una singola immagine LDR (*Low Dynamic Range*) non è sempre sufficiente per rappresentare scene con alto range dinamico. In presenza di regioni molto scure e molto luminose, una sola esposizione tende a perdere informazione: le ombre possono risultare sottoesposte, mentre le alte luci possono essere saturate.

La fotografia HDR classica affronta il problema acquisendo una sequenza di esposizioni e stimando una rappresentazione radiometrica della scena. Questa rappresentazione viene poi trasformata in una immagine visualizzabile tramite tone mapping:

`bracketed exposures -> HDR reconstruction -> tone mapping -> display image`

Exposure Fusion propone una pipeline più diretta. Invece di stimare la luminanza fisica della scena, assegna pesi locali ai pixel delle immagini LDR e li combina in una immagine finale:

`bracketed exposures -> weight maps -> multiresolution blending -> display image`

Il vantaggio principale è la semplificazione: il metodo non richiede tempi di esposizione, calibrazione radiometrica o stima della curva di risposta della camera. Il risultato finale è una immagine LDR visivamente plausibile, non una radiance map HDR.

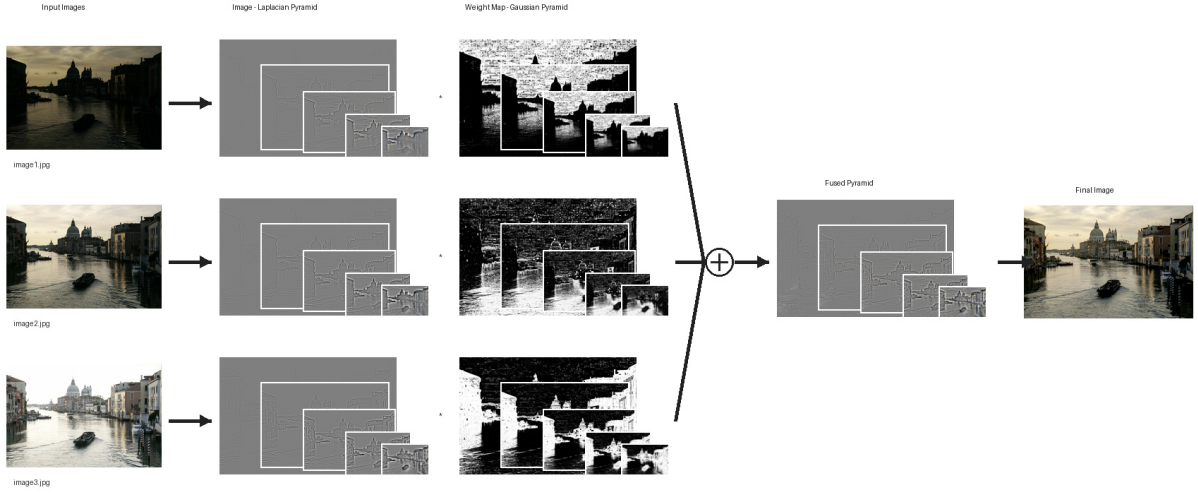


Figura 1: Pipeline completa dell'implementazione sul dataset Venice Boat.

3 Metodo

L'idea di base del paper è che, in una sequenza di esposizioni, ogni immagine contiene regioni localmente più utili di altre. Una esposizione scura può preservare dettagli nelle alte luci; una esposizione chiara può rendere visibili dettagli nelle ombre. L'algoritmo assegna quindi a ogni pixel un peso che misura quanto quel pixel sia adatto a contribuire al risultato finale.

Per ogni pixel (i, j) dell'immagine k , vengono calcolate tre misure:

- **contrasto**, che favorisce bordi, texture e variazioni locali;
- **saturnazione**, che favorisce colori non sbiaditi;
- **well-exposedness**, che favorisce valori di intensità intermedi.

Le tre misure vengono combinate tramite prodotto:

$$W_{i,j,k} = C_{i,j,k}^{\omega_C} S_{i,j,k}^{\omega_S} E_{i,j,k}^{\omega_E}$$

dove C , S ed E indicano contrasto, saturazione e well-exposedness, mentre gli esponenti ω_C , ω_S e ω_E controllano l'importanza relativa delle componenti. Nell'implementazione di default tutti gli esponenti valgono 1.0.

I pesi grezzi vengono poi normalizzati lungo la sequenza:

$$\hat{W}_{i,j,k} = \frac{W_{i,j,k}}{\sum_n W_{i,j,n}}$$

in modo che, per ogni pixel, la somma dei contributi delle diverse esposizioni sia pari a uno:

$$\sum_k \hat{W}_{i,j,k} = 1.$$

Una fusione pixel-wise diretta può essere scritta come:

$$R_{i,j} = \sum_k \hat{W}_{i,j,k} I_{i,j,k}.$$

Questa forma è semplice, ma può produrre transizioni innaturali quando le mappe di peso cambiano bruscamente o quando le esposizioni hanno luminosità globali molto diverse. Per ridurre questi artefatti, il paper combina le immagini in multirisoluzione: le immagini vengono decomposte in piramidi Laplaciane, i pesi in piramidi Gaussiane, e la fusione avviene livello per livello.

4 Struttura dell'implementazione

Il progetto separa il codice in moduli che corrispondono alle fasi dell'algoritmo. La tabella seguente riassume la relazione tra concetti del paper e implementazione.

Concetto	Ruolo nell'algoritmo	Implementazione
Misure di qualità	Contrasto, saturazione, well-exposedness	<code>exposure_fusion/weights.py</code> , <code>quality_measures</code>
Formula dei pesi	Prodotto $C^{\omega_C} S^{\omega_S} E^{\omega_E}$	<code>weight_map</code>
Normalizzazione	Somma pixel-wise dei pesi pari a 1	<code>normalize_weights</code>
Piramidi	Decomposizione Gaussianica e Laplaciana	<code>exposure_fusion/pyramids.py</code>
Fusione	Blending livello per livello	<code>exposure_fusion/fusion.py</code> , <code>fuse_exposures</code>

L'interfaccia principale è `fuse_exposures`. La funzione riceve una lista di immagini RGB con la stessa shape, caricate come array NumPy in virgola mobile con valori in $[0, 1]$, e restituisce una struttura `ExposureFusionResult`. Questa struttura contiene l'immagine finale, ma anche pesi grezzi, pesi normalizzati, piramidi delle immagini, piramidi dei pesi e piramide Laplaciana fusa. La scelta rende la pipeline ispezionabile e utile per uno studio didattico dell'algoritmo.

L'entry point `main.py` permette di eseguire la pipeline da riga di comando e di salvare artefatti intermedi, come input LDR, mappe di peso, confronto con un riferimento e diagramma del processo. Un esempio di uso per rigenerare anche le anteprime aggregate mostrate in questo documento è:

```

uv run python main.py \
  --inputs images/venice_boat/image1.jpg \
    images/venice_boat/image2.jpg \
    images/venice_boat/image3.jpg \
  --output-dir images/venice_boat/out \
  --reference images/venice_boat/result.jpg \
  --save-all \
  --save-process \
  --preview

```

I dataset e gli output previsti sono dichiarati in `paper_sets.toml`. I casi discussi visivamente nel documento sono `venice_boat` e `living_room_window`; il dataset `venice_carnival` è disponibile come ulteriore esempio e viene incluso nei controlli numerici.

5 Calcolo delle mappe di peso

La prima fase della pipeline calcola una mappa di peso per ogni immagine. Le immagini sono convertite in RGB e normalizzate nell'intervallo $[0, 1]$ tramite `load_rgb_float`.

5.1 Contrasto

Il contrasto viene stimato applicando un filtro Laplaciano all'immagine in scala di grigi:

```

gray = rgb2gray(image)
contrast = np.abs(convolve(gray, LAPLACIAN_KERNEL, mode="reflect"))

```

La risposta assoluta del Laplaciano evidenzia bordi e variazioni locali. La gestione dei bordi usa `mode="reflect"`, quindi i valori vicino ai margini dipendono da questa specifica convenzione di convoluzione. Il punto importante è che la misura favorisce dettaglio locale; non va interpretata come una misura radiometrica della scena.

5.2 Saturazione

La saturazione viene stimata come deviazione standard dei canali RGB:

```

saturation = np.std(image, axis=2)

```

Questa è una proxy semplice e locale. Un pixel con canali simili tende ad avere bassa saturazione; un pixel con canali più differenziati tende ad avere una risposta maggiore. Non si tratta di un modello percettivo completo della saturazione colore.

5.3 Well-exposedness

La well-exposedness favorisce intensità intermedie e penalizza valori vicini a 0 o 1. L'implementazione usa una curva Gaussiana centrata in 0.5:

$$E(x) = \exp\left(-\frac{(x - 0.5)^2}{2\sigma^2}\right).$$

La curva viene applicata ai canali RGB e i risultati vengono moltiplicati:

```
well_exposedness = np.prod(
    exposedness_curve(image, sigma=config.sigma_exposedness),
    axis=2,
)
```

Con la configurazione di default, `sigma_exposedness` vale 0.2. Il peso finale è il prodotto delle tre misure:

```
weight = (
    contrast**config.omega_contrast
    * saturation**config.omega_saturation
    * well_exposedness**config.omega_exposedness
)
```

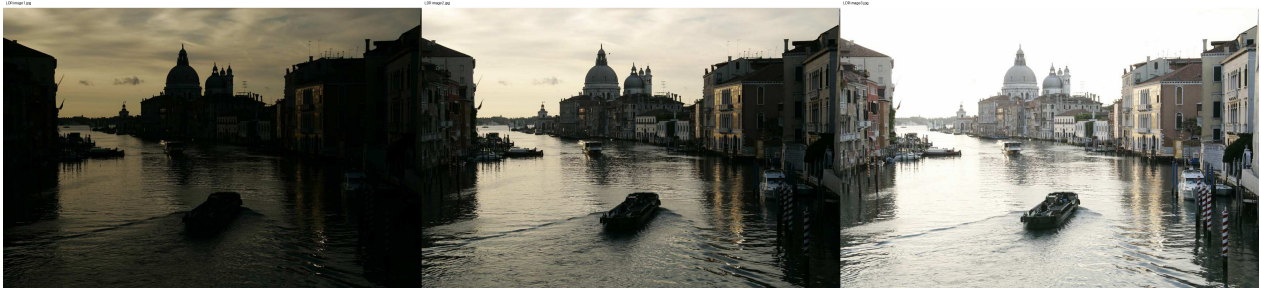


Figura 2: Esposizioni di input per il dataset Venice Boat.

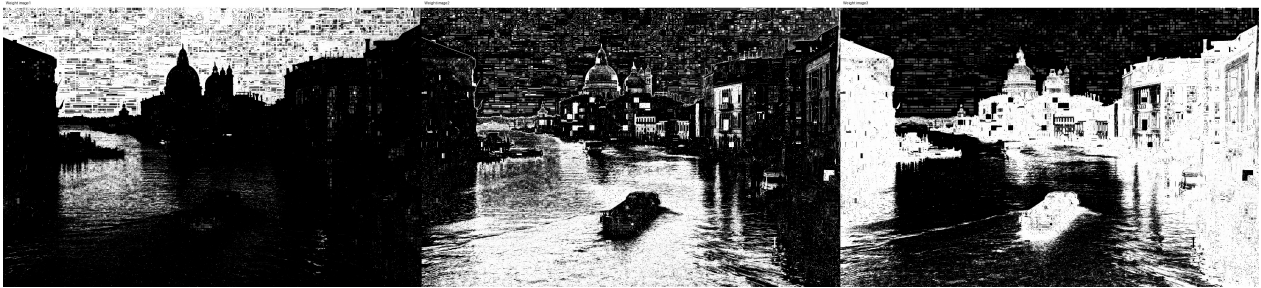


Figura 3: Mappe di peso normalizzate per il dataset Venice Boat.

6 Normalizzazione dei pesi

Dopo il calcolo dei pesi grezzi, le mappe vengono normalizzate lungo la dimensione delle immagini. Se `weights` ha forma:

```
(num_images, height, width)
```

la normalizzazione avviene lungo `axis=0`:

```
weight_sum = weights.sum(axis=0, keepdims=True)
uniform = np.full_like(weights, 1.0 / weights.shape[0])
normalized = np.divide(weights, weight_sum, out=uniform, where=weight_sum > eps)
```

Il fallback uniforme viene usato quando la somma locale dei pesi è nulla o numericamente degenerata. In quel caso, invece di dividere per zero, ogni immagine riceve lo stesso contributo. Questo controllo evita che la fusione sia indefinita nei pixel in cui tutte le misure producono peso nullo.

7 Fusione multirisoluzione

La fusione finale segue la struttura del paper: non combina direttamente i pixel, ma combina coefficienti a scale diverse. Per ogni immagine viene costruita una piramide Laplaciana; per ogni mappa di peso normalizzata viene costruita una piramide Gaussiana.

Per ogni livello l :

$$L_l(R) = \sum_k G_l(\hat{W}_k) \cdot L_l(I_k)$$

dove $L_l(I_k)$ è il livello l della piramide Laplaciana dell'immagine k , $G_l(\hat{W}_k)$ è il livello l della piramide Gaussiana del peso normalizzato, e $L_l(R)$ è il livello fuso. La piramide risultante viene poi collassata per ottenere l'immagine finale.

Le piramidi sono costruite con funzioni di `scikit-image`: `pyramid_gaussian` e `pyramid_expand`. Poiché downsampling e upsampling possono introdurre differenze di shape sui bordi, `crop_like` riallinea l'immagine espansa alla shape del livello di riferimento. Inoltre, dopo aver costruito le piramidi Gaussiani dei pesi, `fuse_exposures` rinormalizza i pesi a ogni livello. Questo riduce piccoli drift numerici dovuti al filtraggio e al ridimensionamento.

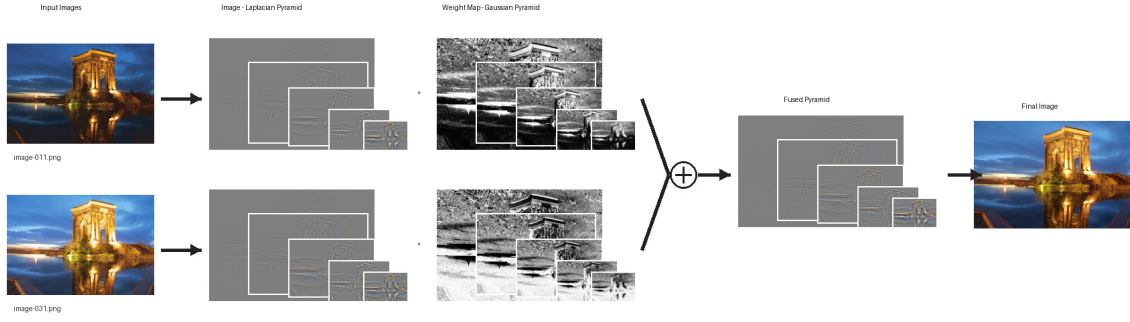


Figura 4: Schema di piramidi Gaussiane, piramidi Laplaciane e fusione multiscala.

8 Dataset e risultati

Gli esperimenti documentati usano i gruppi di immagini dichiarati in `paper_sets.toml`: tre dataset di fusione (`venice_boat`, `venice_carnival` e `living_room_window`) e un dataset di supporto per spiegare la costruzione delle piramidi (`pyramid_blending`). Gli output sono salvati nelle rispettive cartelle `out/`, in modo da mantenere separati input originali e artefatti generati.

8.1 Venice Boat

`venice_boat` contiene tre esposizioni di una scena esterna. Il dataset è utile per mostrare come esposizioni differenti contribuiscano a regioni diverse dell'immagine finale. La figura seguente confronta il risultato della fusione con il riferimento disponibile e con una mappa di differenza assoluta.



Figura 5: Confronto finale per il dataset Venice Boat.

La valutazione resta qualitativa: il riferimento non dimostra una correttezza radiometrica assoluta, ma permette di ispezionare visivamente luminosità, contrasto e differenze locali.

8.2 Venice Carnival

`venice_carnival` contiene tre esposizioni di una scena con un soggetto in maschera in Piazza San Marco. Rispetto a `venice_boat`, il caso presenta colori più saturi e regioni con texture più evidenti; per questo è utile per ispezionare il comportamento combinato di saturazione, contrasto e well-exposedness.



Figura 6: Esposizioni di input per il dataset Venice Carnival.

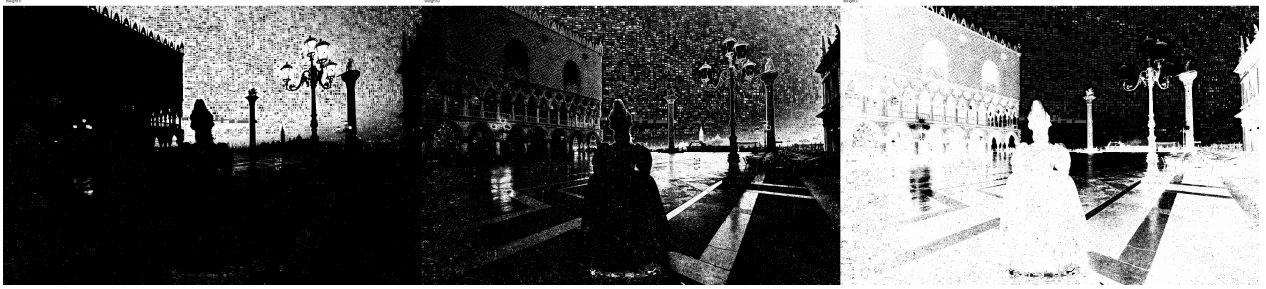


Figura 7: Mappe di peso normalizzate per il dataset Venice Carnival.



Figura 8: Confronto finale per il dataset Venice Carnival.

Nel confronto finale, la fusione conserva il soggetto principale e combina le diverse esposizioni per ridurre sia le zone troppo scure sia le regioni troppo chiare. La scena è anche un buon caso di controllo visivo per eventuali artefatti: colori intensi, bordi della maschera e dettagli dello sfondo rendono più evidente se le mappe di peso o il blending multirisoluzione introducono transizioni innaturali.

8.3 Living Room Window

`living_room_window` contiene quattro immagini di una scena interna con una finestra luminosa. Il caso è rappresentativo perché contiene una forte differenza tra dettagli nelle zone interne scure e regione esterna molto luminosa.

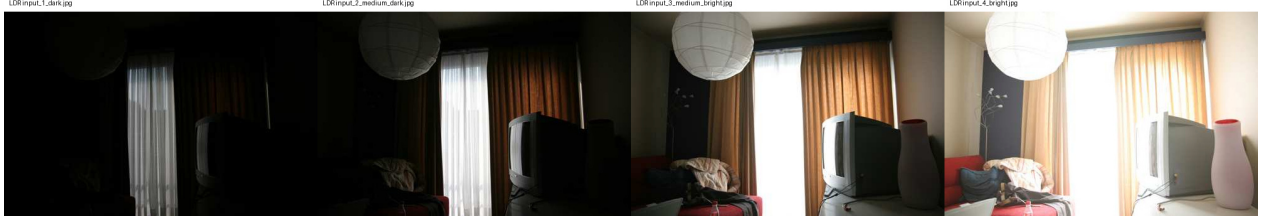


Figura 9: Esposizioni di input per il dataset Living Room Window.

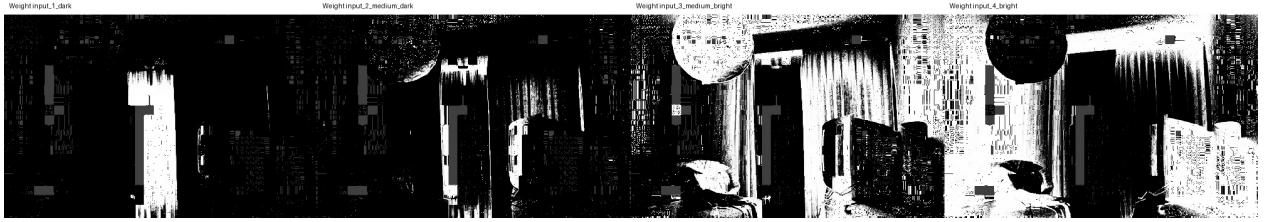


Figura 10: Mappe di peso normalizzate per il dataset Living Room Window.

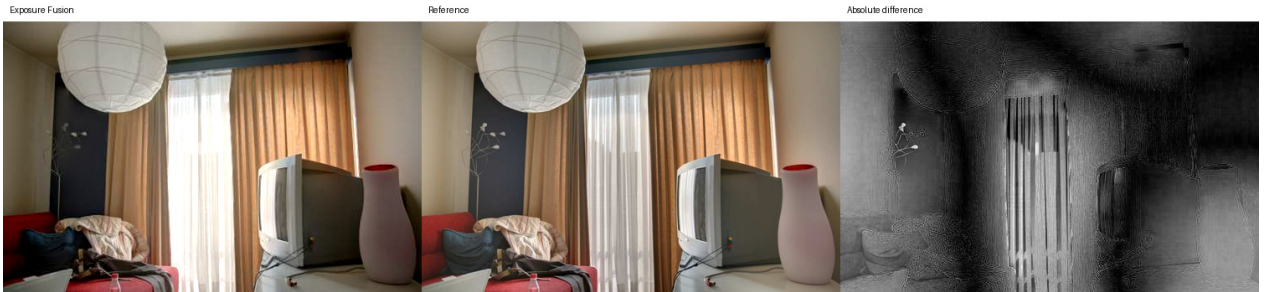


Figura 11: Confronto finale per il dataset Living Room Window.

Le mappe di peso mostrano qualitativamente il ruolo della well-exposedness: le esposizioni scure tendono a contribuire nelle alte luci, mentre quelle più chiare contribuiscono nelle aree interne. La fusione multirisoluzione combina questi contributi cercando transizioni più regolari rispetto a una fusione pixel-wise semplice.

9 Controlli di correttezza

La qualità finale di Exposure Fusion è in parte percettiva, quindi non può essere riassunta da una sola metrica numerica. Tuttavia, alcuni controlli verificano che la struttura matematica dell'impe-

mentazione sia coerente.

9.1 Normalizzazione

Per ogni pixel, la somma dei pesi normalizzati lungo la sequenza deve essere circa uguale a 1:

$$\max_{i,j} \left| \sum_k \hat{W}_{i,j,k} - 1 \right| \approx 0.$$

Il controllo è stato calcolato sui dataset di fusione disponibili. La colonna dei pesi piramidali riporta il massimo errore osservato dopo la rinormalizzazione dei livelli Gaussiani.

Dataset	Livelli	Errore pesi iniziali	Errore pesi piramidali
venice_boat	12	$3.331 \cdot 10^{-16}$	$2.220 \cdot 10^{-16}$
living_room_window	10	$3.331 \cdot 10^{-16}$	$3.331 \cdot 10^{-16}$
venice_carnival	12	$3.331 \cdot 10^{-16}$	$2.220 \cdot 10^{-16}$

9.2 Ricostruzione della piramide Laplaciana

Un secondo controllo verifica che la costruzione e il collasso della piramide Laplaciana preservino l'immagine originale entro la tolleranza numerica:

$$\text{collapse}(\text{laplacian_pyramid}(I)) \approx I.$$

Sulla prima immagine di ciascun dataset, l'errore massimo di ricostruzione è:

Dataset	Errore massimo di ricostruzione
venice_boat	$3.331 \cdot 10^{-16}$
living_room_window	$3.331 \cdot 10^{-16}$
venice_carnival	$2.220 \cdot 10^{-16}$

9.3 Coerenza delle shape

Il terzo controllo verifica che ogni livello Gaussiano dei pesi abbia la stessa altezza e larghezza del corrispondente livello Laplaciano delle immagini. Questa condizione è necessaria per calcolare il prodotto $G_l(\hat{W}_k) \cdot L_l(I_k)$ senza ambiguità dimensionali. Sui dataset verificati, la coerenza delle shape risulta soddisfatta per tutti i livelli.

10 Differenze implementative e limiti

L'implementazione segue la struttura principale del metodo, ma include alcune scelte pratiche:

- le immagini sono rappresentate come array NumPy RGB in $[0, 1]$;
- il contrasto usa un kernel Laplaciano discreto con gestione dei bordi `reflect`;
- la saturazione è stimata come deviazione standard dei canali RGB;
- la well-exposedness usa una Gaussiana centrata a 0.5 con `sigma=0.2`;
- le piramidi sono costruite tramite funzioni di `scikit-image`;
- i pesi vengono rinormalizzati anche ai livelli piramidali;
- l'output finale viene clippato in $[0, 1]$ prima del salvataggio;
- il progetto è orientato allo studio dell'algoritmo, non alla riproduzione bit-exact del codice originale.

Queste scelte non cambiano il nucleo della pipeline, ma vanno dichiarate per evitare claim troppo forti sulla corrispondenza con il paper.

Il metodo assume inoltre che le immagini della sequenza siano allineate. Se la camera o i soggetti si muovono tra gli scatti, la fusione può produrre ghosting o artefatti locali. L'implementazione presentata non include registrazione delle immagini o compensazione del movimento. Anche rumore, clipping estremo, bilanciamento colore incoerente e scelta degli esponenti ω possono influenzare il risultato.

Infine, Exposure Fusion produce direttamente una immagine LDR. Questo è utile quando si vuole evitare la pipeline HDR completa, ma significa che il risultato non rappresenta una radiance map fisicamente calibrata della scena.

11 Conclusioni

Il progetto implementa in Python il nucleo dell'algoritmo Exposure Fusion:

`quality-based pixel weighting + pyramid-based multiresolution blending`

La corrispondenza con il paper è visibile nella struttura matematica e nella suddivisione del codice: `weights.py` calcola le misure di qualità e le mappe di peso, `pyramids.py` gestisce le decomposizioni multirisoluzione e `fusion.py` combina immagini e pesi livello per livello. I controlli numerici mostrano che i pesi sono normalizzati, che le piramidi Laplaciane sono ricostruibili entro la precisione floating point e che i livelli piramidali hanno shape compatibili.

Il risultato è una pipeline semplice, ispezionabile e coerente con il metodo di Mertens, Kautz e Van Reeth. Possibili sviluppi futuri includono test automatici, analisi quantitativa su più dataset, registrazione delle immagini per sequenze non perfettamente allineate e studio sistematico dell'effetto degli esponenti ω sulla qualità finale.

12 Riferimenti

- T. Mertens, J. Kautz, F. Van Reeth, *Exposure Fusion*, Proceedings of the 15th Pacific Conference on Computer Graphics and Applications (Pacific Graphics), 2007.
- `exposure-fusion.pdf`, copia del paper di riferimento inclusa nel repository.
- `sample.ipynb`, notebook di studio e sperimentazione dell'algoritmo.
- `exposure_fusion/`, package Python riusabile dell'implementazione.